CHAPTER

TWENTYSIX

NETWORK PROTOCOLS — BLUETOOTH

26.1 EFI Bluetooth Host Controller Protocol

26.1.1 EFI_BLUETOOTH_HC_PROTOCOL

Summary

This protocol abstracts the Bluetooth host controller layer.message transmit and receive.

GUID

```
#define EFI_BLUETOOTH_HC_PROTOCOL_GUID \
  { 0xb3930571, 0xbeba, 0x4fc5,
    { 0x92, 0x3, 0x94, 0x27, 0x24, 0x2e, 0x6a, 0x43 }}
```

Protocol Interface Structure

```
typedef struct  _EFI_BLUETOOTH_HC_PROTOCOL {
  EFI_BLUETOOTH_HC_SEND_COMMAND               SendCommand;
  EFI_BLUETOOTH_HC_RECEIVE_EVENT              ReceiveEvent;
  EFI_BLUETOOTH_HC_ASYNC_RECEIVE_EVENT        AsyncReceiveEvent;
  EFI_BLUETOOTH_HC_SEND_ACL_DATA              SendACLData;
  EFI_BLUETOOTH_HC_RECEIVE_ACL_DATA           ReceiveACLData;
  EFI_BLUETOOTH_HC_ASYNC_RECEIVE_ACL_DATA     AsyncReceiveACLData;
  EFI_BLUETOOTH_HC_SEND_SCO_DATA              SendSCOData;
  EFI_BLUETOOTH_HC_RECEIVE_SCO_DATA           ReceiveSCOData;
  EFI_BLUETOOTH_HC_ASYNC_RECEIVE_SCO_DATA     AsyncReceiveSCOData;
}   EFI_BLUETOOTH_HC_PROTOCOL;
```

Parameters

SendCommand
Send HCI command packet. See the *SendCommand()* function description.

ReceiveEvent
Receive HCI event packets. See the *ReceiveEvent()* function description.

AsyncReceiveEvent
Non-blocking receive HCI event packets. See the *AsyncReceiveEvent()* function description.

SendACLData
Send HCI ACL (asynchronous connection-oriented) data packets. See the *SendACLData()* function description.

ReceiveACLData
Receive HCI ACL data packets. See the *ReceiveACLData()* function description.

AsyncReceiveACLData
Non-blocking receive HCI ACL data packets. See the *AsyncReceiveACLData()* function description.

SendSCOData
Send HCI synchronous (SCO and eSCO) data packets. See the *SendSCOData()* function description.

ReceiveSCOData
Receive HCI synchronous data packets. See the *ReceiveSCOData()* function description.

AsyncReceiveSCOData
Non-blocking receive HCI synchronous data packets. See the *AsyncReceiveSCOData()* function description.

Description

The EFI_BLUETOOTH_HC_PROTOCOL is used to transmit or receive HCI layer data packets. For detail of different HCI packet (command, event, ACL, SCO), please refer to Bluetooth specification.

26.1.2 BLUETOOTH_HC_PROTOCOL.SendCommand()

Summary

Send HCI command packet.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_SEND_COMMAND)(
  IN EFI_BLUETOOTH_HC_PROTOCOL            *This,
  IN OUT UINTN                            *BufferSize,
  IN VOID                                 *Buffer,
  IN UINTN                                Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be transmitted to Bluetooth host controller.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The SendCommand() function sends HCI command packet. *Buffer* holds the whole HCI command packet, including OpCode, OCF, OGF, parameter length, and parameters. When this function is returned, it just means the HCI command packet is sent, it does not mean the command is success or complete. Caller might need to wait a command status event to know the command status, or wait a command complete event to know if the command is completed. (see in Bluetooth specification, HCI Command Packet for more detail).

Status Codes Returned

EFI_SUCCESS	The HCI command packet is sent successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: *BufferSize* is NULL. * *BufferSize* is 0. *Buffer* is NULL.
EFI_TIMEOUT	Sending HCI command packet fail due to timeout.
EFI_DEVICE_ERROR	Sending HCI command packet fail due to host controller or device error.

26.1.3 BLUETOOTH_HC_PROTOCOL.ReceiveEvent()

Summary

Receive HCI event packet.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_RECEIVE_EVENT)(
  IN EFI_BLUETOOTH_HC_PROTOCOL            *This,
  IN OUT UINTN                            *BufferSize,
  OUT VOID                                *Buffer,
  IN UINTN                                Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be received from Bluetooth host controller.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The ReceiveEvent() function receives HCI event packet. *Buffer* holds the whole HCI event packet, including *EventCode*, parameter length, and parameters. (See in Bluetooth specification, HCI Event Packet for more detail.)

Status Codes Returned

EFI_SUCCESS	The HCI event packet is received successfully.

continues on next page

Table 26.2 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Receiving HCI event packet fail due to timeout.
EFI_DEVICE_ERROR	Receiving HCI event packet fail due to host controller or device error.

26.1.4 BLUETOOTH_HC_PROTOCOL.AsyncReceiveEvent()

Summary

Receive HCI event packet in non-blocking way.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_ASYNC_RECEIVE_EVENT) (
  IN EFI_BLUETOOTH_HC_PROTOCOL                *This,
  IN BOOLEAN                                  IsNewTransfer,
  IN UINTN                                    PollingInterval,
  IN UINTN                                    DataLength,
  IN EFI_BLUETOOTH_HC_ASYNC_FUNC_CALLBACK     Callback,
  IN VOID                                     *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

IsNewTransfer
If *TRUE,* a new transfer will be submitted. If *FALSE,* the request is deleted.

PollingInterval
Indicates the periodic rate, in milliseconds, that the transfer is to be executed.

DataLength
Specifies the length, in bytes, of the data to be received.

Callback
The callback function. This function is called if the asynchronous transfer is completed.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL.*

Description

The AsyncReceiveEvent() function receives HCI event packet in non-blocking way. *Data* in *Callback* function holds the whole HCI event packet, including *EventCode*, parameter length, and parameters. (See in Bluetooth specification, HCI Event Packet for more detail.)

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_ASYNC_FUNC_CALLBACK) (
  IN VOID                       *Data,
  IN UINTN                      DataLength,
  IN VOID                       *Context
  );
```

Data
Data received via asynchronous transfer.

DataLength
The length of *Data* in bytes, received via asynchronous transfer.

Context
Context passed from asynchronous transfer request.

Status Codes Returned

EFI_SUCCESS	The HCI asynchronous receive request is submitted successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataLength* is 0. • If *IsNewTransfer* is *TRUE,* and an asynchronous receive request already exists.

26.1.5 BLUETOOTH_HC_PROTOCOL.SendACLData()

Summary

Send HCI ACL data packet.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_SEND_ACL_DATA)(
  IN EFI_BLUETOOTH_HC_PROTOCOL   *This,
  IN OUT UINTN                   *BufferSize,
  IN VOID                        *Buffer,
  IN UINTN                       Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be transmitted to Bluetooth host controller.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The SendACLData() function sends HCI ACL data packet. *Buffer* holds the whole HCI ACL data packet, including Handle, PB flag, BC flag, data length, and data. (See in Bluetooth specification, HCI ACL Data Packet for more detail.)

The SendACLData() function and *ReceiveACLData()* function just send and receive data payload from application layer. In order to protect the payload data, the Bluetooth bus is required to call HCI_Set_Connection_Encryption command to enable hardware-based encryption after authentication completed, according to pairing mode and host capability.

Status Codes Returned

EFI_SUCCESS	The HCI ACL data packet is sent successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Sending HCI ACL data packet fail due to timeout.
EFI_DEVICE_ERROR	Sending HCI ACL data packet fail due to host controller or device error.

26.1.6 BLUETOOTH_HC_PROTOCOL.ReceiveACLData()

Summary

Receive HCI ACL data packet.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_RECEIVE_ACL_DATA)(
  IN EFI_BLUETOOTH_HC_PROTOCOL         *This,
  IN OUT UINTN                         *BufferSize,
  OUT VOID                             *Buffer,
  IN UINTN                             Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be received from Bluetooth host controller.

Timeout

Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The ReceiveACLData() function receives HCI ACL data packet. *Buffer* holds the whole HCI ACL data packet, including Handle, PB flag, BC flag, data length, and data. (See in Bluetooth specification, HCI ACL Data Packet for more detail.)

Status Codes Returned

EFI_SUCCESS	The HCI ACL data packet is received successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Receiving HCI ACL data packet fail due to timeout.
EFI_DEVICE_ERROR	Receiving HCI ACL data packet fail due to host controller or device error.

26.1.7 BLUETOOTH_HC_PROTOCOL.AsyncReceiveACLData()

Summary

Receive HCI ACL data packet in non-blocking way.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_ASYNC_RECEIVE_ACL_DATA) (
  IN EFI_BLUETOOTH_HC_PROTOCOL                *This,
  IN BOOLEAN                                  IsNewTransfer,
  IN UINTN                                    PollingInterval,
  IN UINTN                                    DataLength,
  IN EFI_BLUETOOTH_HC_ASYNC_FUNC_CALLBACK     Callback,
  IN VOID                                     *Context
  );
```

Parameters

This

Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

IsNewTransfer

If *TRUE,* a new transfer will be submitted.

If *FALSE,* the request is deleted.

PollingInterval

Indicates the periodic rate, in milliseconds, that the transfer is to be executed.

DataLength
Specifies the length, in bytes, of the data to be received.

Callback
The callback function. This function is called if the asynchronous transfer is completed.

Context
Data passed into *Callback* function. This is optional parameter and may be NULL.

Description

The AsyncReceiveACLData() function receives HCI ACL data packet in non-blocking way. *Data* in *Callback* holds the whole HCI ACL data packet, including Handle, PB flag, BC flag, data length, and data. (See in Bluetooth specification, HCI ACL Data Packet for more detail.)

Status Codes Returned

EFI_SUCCESS	The HCI asynchronous receive request is submitted successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataLength* is 0. • If *IsNewTransfer* is *TRUE,* and an asynchronous receive request already exists.

26.1.8 BLUETOOTH_HC_PROTOCOL.SendSCOData()

Summary

Send HCI SCO data packet.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_SEND_SCO_DATA)(
  IN EFI_BLUETOOTH_HC_PROTOCOL            *This,
  IN OUT UINTN                            *BufferSize,
  IN VOID                                 *Buffer,
  IN UINTN                                Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be transmitted to Bluetooth host controller.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is

0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The SendSCOData() function sends HCI SCO data packet. *Buffer* holds the whole HCI SCO data packet, including *ConnectionHandle*, *PacketStatus* flag, data length, and data. (See in Bluetooth specification, HCI Synchronous Data Packet for more detail.)

Status Codes Returned

EFI_SUCCESS	The HCI SCO data packet is sent successfully.
EFI_UNSUPPORTED	The implementation does not support HCI SCO transfer.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Sending HCI SCO data packet fail due to timeout.
EFI_DEVICE_ERROR	Sending HCI SCO data packet fail due to host controller or device error.

26.1.9 BLUETOOTH_HC_PROTOCOL.ReceiveSCOData()

Summary

Receive HCI SCO data packet.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_RECEIVE_SCO_DATA)(
  IN EFI_BLUETOOTH_HC_PROTOCOL         *This,
  IN OUT UINTN                         *BufferSize,
  OUT VOID                             *Buffer,
  IN UINTN                             Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be received from Bluetooth host controller.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The ReceiveSCOData() function receives HCI SCO data packet. *Buffer* holds the whole HCI SCO data packet, including ConnectionHandle, PacketStatus flag, data length, and data. (See in Bluetooth specification, HCI Synchronous Data Packet for more detail.)

Status Codes Returned

EFI_SUCCESS	The HCI SCO data packet is received successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Receiving HCI SCO data packet fail due to timeout.
EFI_DEVICE_ERROR	Receiving HCI SCO data packet fail due to host controller or device error.

26.1.10 BLUETOOTH_HC_PROTOCOL.AsyncReceiveSCOData()

Summary

Receive HCI SCO data packet in non-blocking way.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_HC_ASYNC_RECEIVE_SCO_DATA) (
  IN EFI_BLUETOOTH_HC_PROTOCOL               *This,
  IN BOOLEAN                                 IsNewTransfer,
  IN UINTN                                   PollingInterval,
  IN UINTN                                   DataLength,
  IN EFI_BLUETOOTH_HC_ASYNC_FUNC_CALLBACK    Callback,
  IN VOID                                    *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_HC_PROTOCOL* instance.

IsNewTransfer
If *TRUE,* a new transfer will be submitted. If *FALSE,* the request is deleted.

PollingInterval
Indicates the periodic rate, in milliseconds, that the transfer is to be executed.

DataLength
Specifies the length, in bytes, of the data to be received.

Callback
The callback function. This function is called if the asynchronous transfer is completed.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL.*

Description

The AsyncReceiveSCOData() function receives HCI SCO data packet in non-blocking way. *Data* in *Callback* holds the whole HCI SCO data packet, including *ConnectionHandle*, *PacketStatus* flag, data length, and data. (See in Bluetooth specification, HCI SCO Data Packet for more detail.)

Status Codes Returned

EFI_SUCCESS	The HCI asynchronous receive request is submitted successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataLength* is 0. • If *IsNewTransfer* is *TRUE,* and an asynchronous receive request already exists.

26.2 EFI Bluetooth Bus Protocol

26.2.1 EFI_BLUETOOTH_IO_SERVICE_BINDING_PROTOCOL

Summary

The EFI Bluetooth IO Service Binding Protocol is used to locate EFI Bluetooth IO Protocol drivers to create and destroy child of the driver to communicate with other Bluetooth device by using Bluetooth IO protocol.

GUID

```
#define EFI_BLUETOOTH_IO_SERVICE_BINDING_PROTOCOL_GUID \
  { 0x388278d3, 0x7b85, 0x42f0,\
    { 0xab, 0xa9, 0xfb, 0x4b, 0xfd, 0x69, 0xf5, 0xab }
```

Description

The Bluetooth IO consumer need locate *EFI_BLUETOOTH_IO_SERVICE_BINDING_PROTOCOL* and call *CreateChild()* to create a new child of *EFI_BLUETOOTH_IO_PROTOCOL* instance. Then use *EFI_BLUETOOTH_IO_PROTOCOL* for Bluetooth communication. After use, the Bluetooth IO consumer need call *DestroyChild()* to destroy it.

26.2.2 EFI_BLUETOOTH_IO_PROTOCOL

Summary

This protocol provides service for Bluetooth L2CAP (Logical Link Control and Adaptation Protocol) and SDP (Service Discovery Protocol).

GUID

```
#define EFI_BLUETOOTH_IO_PROTOCOL_GUID \
  { 0x467313de, 0x4e30, 0x43f1,\
    { 0x94, 0x3e, 0x32, 0x3f, 0x89, 0x84, 0x5d, 0xb5 }}
```

Protocol Interface Structure

```
typedef struct  _EFI_BLUETOOTH_IO_PROTOCOL {
  EFI_BLUETOOTH_IO_GET_DEVICE_INFO           GetDeviceInfo;
  EFI_BLUETOOTH_IO_GET_SDP_INFO              GetSdpInfo;
  EFI_BLUETOOTH_IO_L2CAP_RAW_SEND            L2CapRawSend;
  EFI_BLUETOOTH_IO_L2CAP_RAW_RECEIVE         L2CapRawReceive;
  EFI_BLUETOOTH_IO_L2CAP_RAW_ASYNC_RECEIVE\  L2CapRawAsyncReceive;
  EFI_BLUETOOTH_IO_L2CAP_SEND                L2CapSend;
  EFI_BLUETOOTH_IO_L2CAP_RECEIVE             L2CapReceive;
  EFI_BLUETOOTH_IO_L2CAP_ASYNC_RECEIVE       L2CapAsyncReceive;
  EFI_BLUETOOTH_IO_L2CAP_CONNECT             L2CapConnect;
  EFI_BLUETOOTH_IO_L2CAP_DISCONNECT          L2CapDisconnect;
  EFI_BLUETOOTH_IO_L2CAP_REGISTER_SERVICE\   L2CapRegisterService;
}
EFI_BLUETOOTH_IO_PROTOCOL;
```

Parameters

GetDeviceInfo
Get Bluetooth device Information. See theGetDeviceInfo()* function description.

GetSdpInfo
Get Bluetooth device SDP information. See the *GetSdpInfo()* function description.

L2CapRawSend
Send L2CAP message (including L2CAP header). See the *L2CapRawSend()* function description.

L2CapRawReceive
Receive L2CAP message (including L2CAP header). See the *L2CapRawReceive()* function description.

L2CapRawAsyncReceive
Non-blocking receive L2CAP message (including L2CAP header). See the *L2CapRawAsyncReceive()* function description.

L2CapSend
Send L2CAP message (excluding L2CAP header) to a specific channel. See the *L2CapSend()* function description.

L2CapReceive
Receive L2CAP message (excluding L2CAP header) from a specific channel. See the *L2CapRawReceive()* function description.

L2CapAsyncReceive
Non-blocking receive L2CAP message (excluding L2CAP header) from a specific channel. See the L2CapRawAsyncReceive() function description.

L2CapConnect
Do L2CAP connection. See the *L2CapConnect()* function description.

L2CapDisconnect
Do L2CAP disconnection. See the *L2CapDisconnect()* function description.

L2CapRegisterService
Register L2CAP callback function for special channel. See the *L2CapRegisterService()* function description.

Description

The *EFI_BLUETOOTH_IO_PROTOCOL* provides services in L2CAP protocol and SDP protocol. For detail of L2CAP packet format, and SDP service, please refer to Bluetooth specification.

26.2.3 BLUETOOTH_IO_PROTOCOL.GetDeviceInfo

Summary

Get Bluetooth device information.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_GET_DEVICE_INFO)(
  IN EFI_BLUETOOTH_IO_PROTOCOL         *This,
  OUT UINTN                            *DeviceInfoSize,
  OUT VOID                             **DeviceInfo
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

DeviceInfoSize
A pointer to the size, in bytes, of the *DeviceInfo* buffer.

DeviceInfo
A pointer to a callee allocated buffer that returns Bluetooth device information. Callee allocates this buffer by using EFI Boot Service *AllocatePool()*.

Description

The GetDeviceInfo() function returns Bluetooth device information. The size of *DeviceInfo* structure should never be assumed and the value of *DeviceInfoSize* is the only valid way to know the size of *DeviceInfo*.

Related Definitions

```
typedef struct {
  UINT32                     Version;
  BLUETOOTH_ADDRESS          BD_ADDR;
  UINT8                      PageScanRepetitionMode;
  BLUETOOTH_CLASS_OF_DEVICE  ClassOfDevice;
  UINT16                     ClockOffset;
  UINT8                      RSSI;
  UINT8                      ExtendedInquiryResponse [240];
}  EFI_BLUETOOTH_DEVICE_INFO;
```

Version
The version of the structure. A value of zero represents the *EFI_BLUETOOTH_DEVICE_INFO* structure as defined here. Future version of this specification may extend this data structure in a backward compatible way and increase the value of *Version*.

BD_ADDR
48bit Bluetooth device address.

PageScanRepetitionMode*
Bluetooth PageScanRepetitionMode. See Bluetooth specification for detail.

ClassOfDevice
Bluetooth ClassOfDevice. See Bluetooth specification for detail.

ClockOffset
Bluetooth CloseOffset. See Bluetooth specification for detail.

RSSI
Bluetooth RSSI. See Bluetooth specification for detail.

ExtendedInquiryResponse
Bluetooth ExtendedInquiryResponse. See Bluetooth specification for detail.

```
typedef struct {
  UINT8              Address [6];
} BLUETOOTH_ADDRESS;

typedef struct {
  UINT8              FormatType:2;
  UINT8              MinorDeviceClass:6;
  UINT16             MajorDeviceClass:5;
  UINT16             MajorServiceClass:11;
}   BLUETOOTH_CLASS_OF_DEVICE;
```

Status Codes Returned

EFI_SUCCESS	The Bluetooth device information is returned successfully.
EFI_DEVICE_ERROR	A hardware error occurred trying to retrieve the Bluetooth device information.

26.2.4 BLUETOOTH_IO_PROTOCOL.GetSdpInfo

Summary

Get Bluetooth SDP information.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_GET_SDP_INFO)(
  IN EFI_BLUETOOTH_IO_PROTOCOL         *This,
  OUT UINTN                            *SdpInfoSize,
  OUT VOID                             **SdpInfo
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

SdpInfoSize
A pointer to the size, in bytes, of the *SdpInfo* buffer.

SdpInfo
A pointer to a callee allocated buffer that returns Bluetooth SDP information. Callee allocates this buffer by using EFI Boot Service *AllocatePool()*.

Description

The GetSdpInfo() function returns Bluetooth SDP information. The size of *SdpInfo* structure should never be assumed and the value of *SdpInfoSize* is the only valid way to know the size of *SdpInfo*.

Status Codes Returned

EFI_SUCCESS	The Bluetooth SDP information is returned successfully.
EFI_DEVICE_ERROR	A hardware error occurred trying to retrieve the Bluetooth SDP information.

26.2.5 BLUETOOTH_IO_PROTOCOL.L2CapRawSend

Summary

Send L2CAP message (including L2CAP header).

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_RAW_SEND)(
  IN EFI_BLUETOOTH_IO_PROTOCOL         *This,
  IN OUT UINTN                         *BufferSize,
  IN VOID                              *Buffer,
  IN UINTN                             Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be transmitted to Bluetooth L2CAP layer.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If Timeout is 0, then the caller must wait for the function to be completed until EFI_SUCCESS or EFI_DEVICE_ERROR is returned.

Description

EFI iSCSI Initiator Name Protocol sends L2CAP layer message (including L2CAP header). Buffer holds the whole L2CAP message, including Length, ChannelID, and information payload. (See the Bluetooth specification, L2CAP Data Packet Format for more details.)

Status Codes Returned

EFI_SUCCESS	The L2CAP message is sent successfully.

continues on next page

Table 26.12 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • BufferSize is NULL. • BufferSize is 0. • Buffer is NULL.
EFI_TIMEOUT	Sending L2CAP message fail due to timeout.
EFI_DEVICE_ERROR	Sending L2CAP message fail due to host controller or device error.

26.2.6 BLUETOOTH_IO_PROTOCOL.L2CapRawReceive

Summary

Receive L2CAP message (including L2CAP header).

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_RAW_RECEIVE)(
  IN EFI_BLUETOOTH_IO_PROTOCOL                *This,
  IN OUT UINTN                                *BufferSize,
  OUT VOID                                    *Buffer,
  IN UINTN                                    Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be received from Bluetooth L2CAP layer.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The L2CapRawReceive() function receives L2CAP layer message (including L2CAP header). *Buffer* holds the whole L2CAP message, including Length, ChannelID, and information payload. (See in Bluetooth specification, L2CAP Data Packet Format for more detail.)

Status Codes Returned

EFI_SUCCESS	The L2CAP message is received successfully.

continues on next page

Table 26.13 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Receiving L2CAP message fail due to timeout.
EFI_DEVICE_ERROR	Receiving L2CAP message fail due to host controller or device error.

26.2.7 BLUETOOTH_IO_PROTOCOL.L2CapRawAsyncReceive

Summary

Receive L2CAP message (including L2CAP header) in non-blocking way.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_RAW_ASYNC_RECEIVE)(
  IN EFI_BLUETOOTH_IO_PROTOCOL               *This,
  IN BOOLEAN                                 IsNewTransfer,
  IN UINTN                                   PollingInterval,
  IN UINTN                                   DataLength,
  IN EFI_BLUETOOTH_IO_ASYNC_FUNC_CALLBACK    Callback,
  IN VOID                                    *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

IsNewTransfer

If *TRUE,* a new transfer will be submitted.
If *FALSE,* the request is deleted.

PollingInterval
Indicates the periodic rate, in milliseconds, that the transfer is to be executed.

DataLength
Specifies the length, in bytes, of the data to be received.

Callback
The callback function. This function is called if the asynchronous transfer is completed.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL.*

Description

The L2CapRawAsyncReceive() function receives L2CAP layer message (including L2CAP header) in non-blocking way. *Data* in *Callback* function holds the whole L2CAP message, including Length, ChannelID, and information payload. (See in Bluetooth specification, L2CAP Data Packet Format for more detail.)

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_ASYNC_FUNC_CALLBACK) (
  IN UINT16                          ChannelID,
  IN VOID                            *Data,
  IN UINTN                           DataLength,
  IN VOID                            *Context
  );
```

ChannelID
Bluetooth L2CAP message channel ID.

Data
Data received via asynchronous transfer.

DataLength
The length of *Data* in bytes, received via asynchronous transfer.

Context
Context passed from asynchronous transfer request.

Status Codes Returned

EFI_SUCCESS	The L2CAP asynchronous receive request is submitted successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataLength* is 0. • If *IsNewTransfer* is *TRUE,* and an asynchronous receive request already exists.

26.2.8 BLUETOOTH_IO_PROTOCOL.L2CapSend

Summary

Send L2CAP message (excluding L2CAP header) to a specific channel.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_SEND)(
  IN EFI_BLUETOOTH_IO_PROTOCOL         *This,
  IN EFI_HANDLE                        Handle,
  IN OUT UINTN                         *BufferSize,
  IN VOID                              *Buffer,
  IN UINTN                             Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

Handle
A handle created by *EFI_BLUETOOTH_IO_PROTOCOL.L2CapConnect* indicates which channel to send.

BufferSize
On input, indicates the size, in bytes, of the data buffer specified by *Buffer*. On output, indicates the amount of data actually transferred.

Buffer
A pointer to the buffer of data that will be transmitted to Bluetooth L2CAP layer.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The L2CapSend() function sends L2CAP layer message (excluding L2CAP header) to Bluetooth channel indicated by *Handle*. *Buffer* only holds information payload. (See in Bluetooth specification, L2CAP Data Packet Format for more detail.)

Status Codes Returned

EFI_SUCCESS	The L2CAP message is sent successfully.
EFI_NOT_FOUND	*Handle* is invalid or not found.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Sending L2CAP message fail due to timeout.
EFI_DEVICE_ERROR	Sending L2CAP message fail due to host controller or device error.

26.2.9 BLUETOOTH_IO_PROTOCOL.L2CapReceive

Summary

Receive L2CAP message (excluding L2CAP header) from a specific channel.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_RECEIVE)(
  IN EFI_BLUETOOTH_IO_PROTOCOL         *This,
  IN EFI_HANDLE                        Handle,
  OUT UINTN                            *BufferSize,
  OUT VOID                             **Buffer,
  IN UINTN                             Timeout
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

Handle
A handle created by *EFI_BLUETOOTH_IO_PROTOCOL.L2CapConnect* indicates which channel to receive.

BufferSize
Indicates the size, in bytes, of the data buffer specified by *Buffer*.

Buffer
A pointer to the buffer of data that will be received from Bluetooth L2CAP layer. Callee allocates this buffer by using EFI Boot Service *AllocatePool()*.

Timeout
Indicating the transfer should be completed within this time frame. The units are in milliseconds. If *Timeout* is 0, then the caller must wait for the function to be completed until *EFI_SUCCESS* or *EFI_DEVICE_ERROR* is returned.

Description

The L2CapReceive() function receives L2CAP layer message (excluding L2CAP header) from Bluetooth channel indicated by *Handle*. *Buffer* only holds information payload. (See in Bluetooth specification, L2CAP Data Packet Format for more detail.)

Status Codes Returned

EFI_SUCCESS	The L2CAP message is received successfully.
EFI_NOT_FOUND	*Handle* is invalid or not found.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *BufferSize* is NULL. • * *BufferSize* is 0. • *Buffer* is NULL.
EFI_TIMEOUT	Receiving L2CAP message fail due to timeout.
EFI_DEVICE_ERROR	Receiving L2CAP message fail due to host controller or device error.

26.2.10 BLUETOOTH_IO_PROTOCOL.L2CapAsyncReceive

Summary

Receive L2CAP message (including L2CAP header) in non-blocking way from a specific channel.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_ASYNC_RECEIVE)(
  IN EFI_BLUETOOTH_IO_PROTOCOL                  *This,
  IN EFI_HANDLE                                 Handle,
  IN EFI_BLUETOOTH_IO_CHANNEL_SERVICE_CALLBACK  Callback,
  IN VOID                                       *Context
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

Handle
A handle created by *EFI_BLUETOOTH_IO_PROTOCOL.L2CapConnect* indicates which channel to receive.

Callback
The callback function. This function is called if the asynchronous transfer is completed.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The L2CapAsyncReceive() function receives L2CAP layer message (excluding L2CAP header) in non-blocking way from Bluetooth channel indicated by *Handle*. *Data* in *Callback* function only holds information payload. (See in Bluetooth specification, L2CAP Data Packet Format for more detail.)

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_CHANNEL_SERVICE_CALLBACK) (
  IN VOID                    *Data,
  IN UINTN                   DataLength,
  IN VOID                    *Context
  );
```

Data
Data received via asynchronous transfer.

DataLength
The length of *Data* in bytes, received via asynchronous transfer.

Context
Context passed from asynchronous transfer request.

Status Codes Returned

EFI_SUCCESS	The L2CAP asynchronous receive request is submitted successfully.
EFI_NOT_FOUND	*Handle* is invalid or not found.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataLength* is 0. • If an asynchronous receive request already exists on same *Handle*.

26.2.11 BLUETOOTH_IO_PROTOCOL.L2CapConnect

Summary

Do L2CAP connection.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_CONNECT)(
  IN EFI_BLUETOOTH_IO_PROTOCOL                  *This,
```

(continues on next page)

(continued from previous page)

```
  OUT EFI_HANDLE                                *Handle,
  IN UINT16                                     Psm,
  IN UINT16                                     Mtu,
  IN EFI_BLUETOOTH_IO_CHANNEL_SERVICE_CALLBACK  Callback,
  IN VOID                                       *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

Handle
A handle to indicate this L2CAP connection.

Psm
Bluetooth PSM. See Bluetooth specification for detail.

Mtu
Bluetooth MTU. See Bluetooth specification for detail.

Callback
The callback function. This function is called whenever there is message received in this channel.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The L2CapConnect() function does all necessary steps for Bluetooth L2CAP layer connection in blocking way. It might take long time. Once this function is returned *Handle* is created to indicate the connection.

Status Codes Returned

EFI_SUCCESS	The Bluetooth L2CAP layer connection is created successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *Handle* is NULL.
EFI_DEVICE_ERROR	A hardware error occurred trying to do Bluetooth L2CAP connection.

26.2.12 BLUETOOTH_IO_PROTOCOL.L2CapDisconnect

Summary

Do L2CAP disconnection.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_DISCONNECT)(
  IN EFI_BLUETOOTH_IO_PROTOCOL     *This,
  IN EFI_HANDLE                    Handle
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

Handle
A handle to indicate this L2CAP connection.

Description

The L2CapDisconnect() function does all necessary steps for Bluetooth L2CAP layer disconnection in blocking way. It might take long time. Once this function is returned *Handle* is no longer valid.

Status Codes Returned

EFI_SUCCESS	The Bluetooth L2CAP layer disconnection is created successfully.
EFI_NOT_FOUND	*Handle* is invalid or not found.
EFI_DEVICE_ERROR	A hardware error occurred trying to do Bluetooth L2CAP disconnection.

26.2.13 BLUETOOTH_IO_PROTOCOL.L2CapRegisterService

Summary

Register L2CAP callback function for special channel.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_IO_L2CAP_REGISTER_SERVICE)(
  IN EFI_BLUETOOTH_IO_PROTOCOL                  *This,
  OUT EFI_HANDLE                                *Handle,
  IN UINT16                                     Psm,
  IN UINT16                                     Mtu,
  IN EFI_BLUETOOTH_IO_CHANNEL_SERVICE_CALLBACK  Callback,
  IN VOID                                       *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_IO_PROTOCOL* instance.

Handle

Psm
Bluetooth PSM. See Bluetooth specification for detail.

Mtu
Bluetooth MTU. See Bluetooth specification for detail.

Callback
The callback function. This function is called whenever there is message received in this channel. *NULL* means unregister.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The L2CapRegisterService() function registers L2CAP callback function for a special channel. Once this function is returned *Handle* is created to indicate the connection.

Status Codes Returned

EFI_SUCCESS	The Bluetooth L2CAP callback function is registered successfully.
EFI_ALREADY_STARTED	The callback function already exists when register.
EFI_NOT_FOUND	The callback function does not exist when unregister.

26.3 EFI Bluetooth Configuration Protocol

26.3.1 EFI_BLUETOOTH_CONFIG_PROTOCOL

Summary

This protocol abstracts user interface configuration for Bluetooth device.

GUID

```
#define EFI_BLUETOOTH_CONFIG_PROTOCOL_GUID \
  { 0x62960cf3, 0x40ff, 0x4263,\
    { 0xa7, 0x7c, 0xdf, 0xde, 0xbd, 0x19, 0x1b, 0x4b }}
```

Protocol Interface Structure

```
typedef struct  _EFI_BLUETOOTH_CONFIG_PROTOCOL {
  EFI_BLUETOOTH_CONFIG_INIT                  Init;
  EFI_BLUETOOTH_CONFIG_SCAN                  Scan;
  EFI_BLUETOOTH_CONFIG_CONNECT               Connect;
  EFI_BLUETOOTH_CONFIG_DISCONNECT            Disconnect;
  EFI_BLUETOOTH_CONFIG_GET_DATA              GetData;
  EFI_BLUETOOTH_CONFIG_SET_DATA              SetData;
  EFI_BLUETOOTH_CONFIG_GET_REMOTE_DATA       GetRemoteData;
EFI_BLUETOOTH_CONFIG_REGISTER_PIN_CALLBACK                RegisterPinCallback;
EFI_BLUETOOTH_CONFIG_REGISTER_GET_LINK_KEY_CALLBACK       RegisterGetLinkKeyCallback;
EFI_BLUETOOTH_CONFIG_REGISTER_SET_LINK_KEY_CALLBACK       RegisterSetLinkKeyCallback;
EFI_BLUETOOTH_CONFIG_REGISTER_CONNECT_COMPLETE_CALLBACK   ␣
↪RegisterLinkConnectCompleteCallback;
} EFI_BLUETOOTH_CONFIG_PROTOCOL;
```

Parameters

Init
Initialize Bluetooth host controller and local device. See the *Init()* function description.

Scan
Scan Bluetooth device. See the *Scan()* function description.

Connect
Connect one Bluetooth device. See the *Connect()* function description.

Disconnect
Disconnect one Bluetooth device. See the *Disconnect()* function description.

GetData
Get Bluetooth configuration data. See the *GetData()* function description.

SetData
Set Bluetooth configuration data. See the *SetData()* function description.

GetRemoteData
Get remote Bluetooth device data. See the *GetRemoteData()* function description.

RegisterPinCallback
Register PIN callback function. See the *RegisterPinCallback()* function description.

RegisterGetLinkKeyCallback
Register get link key callback function. See the *RegisterGetLinkKeyCallback()* function description.

RegisterSetLinkKeyCallback
Register set link key callback function. See the *RegisterSetLinkKeyCallback()* function description.

RegisterLinkConnectCompleteCallback
Register link connect complete callback function. See the *RegisterLinkConnectCompleteCallback()* function description.

Description

The EFI_BLUETOOTH_CONFIG_PROTOCOL abstracts the Bluetooth configuration. User can use Bluetooth configuration to interactive with Bluetooth bus driver.

26.3.2 BLUETOOTH_CONFIG_PROTOCOL.Init

Summary

Initialize Bluetooth host controller and local device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_INIT)(
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL     *This
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Description

The *Init()* function initializes Bluetooth host controller and local device.

Status Codes Returned

EFI_SUCCESS	The Bluetooth host controller and local device is initialized successfully.
EFI_DEVICE_ERROR	A hardware error occurred trying to initialize the Bluetooth host controller and local device.

26.3.3 BLUETOOTH_CONFIG_PROTOCOL.Scan

Summary

Scan Bluetooth device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_SCAN)(
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL                *This,
  IN BOOLEAN                                      ReScan,
  IN UINT8                                        ScanType,
  IN EFI_BLUETOOTH_CONFIG_SCAN_CALLBACK_FUNCTION  Callback
  IN VOID                                         *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

ReScan
If *TRUE,* a new scan request is submitted no matter there is scan result before. If *FALSE* and there is scan result, the previous scan result is returned and no scan request is submitted.

ScanType
Bluetooth scan type, Inquiry and/or Page. See Bluetooth specification for detail.

Callback
The callback function. This function is called if a Bluetooth device is found during scan process.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL.*

Description

The Scan() function scans Bluetooth device. When this function is returned, it just means scan request is submitted. It does not mean scan process is started or finished. Whenever there is a Bluetooth device is found, the *Callback* function will be called. *Callback* function might be called before this function returns or after this function returns.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_SCAN_CALLBACK_FUNCTION) (
IN EFI_BLUETOOTH_CONFIG_PROTOCOL                *This,
IN VOID                                         *Context,
IN EFI_BLUETOOTH_SCAN_CALLBACK_INFORMATION      *CallbackInfo
);
```

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Context
Context passed from scan request.

CallbackInfo
Data related to scan result. *NULL CallbackInfo* means scan complete.

```
typedef
typedef struct{
  BLUETOOTH_ADDRESS                     BDAddr;
  UINT8                                 RemoteDeviceState;
  BLUETOOTH_CLASS_OF_DEVICE             ClassOfDevice;
  UINT8                                 RemoteDeviceName[BLUETOOTH_HCI_COMMAND_LOCAL_
↪READABLE_NAME_MAX_SIZE];
}  EFI_BLUETOOTH_SCAN_CALLBACK_INFORMATION;

#define BLUETOOTH_HCI_COMMAND_LOCAL_READABLE_NAME_MAX_SIZE 248
```

Status Codes Returned

EFI_SUCCESS	The Bluetooth scan request is submitted.
EFI_DEVICE_ERROR	A hardware error occurred trying to scan the Bluetooth device.

26.3.4 BLUETOOTH_CONFIG_PROTOCOL.Connect

Summary

Connect a Bluetooth device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_CONNECT)(
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL     *This,
  IN BLUETOOTH_ADDRESS                 *BD_ADDR
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

BD_ADDR
The address of Bluetooth device to be connected.

Description

The Connect() function connects a Bluetooth device. When this function is returned successfully, a new *EFI_BLUETOOTH_IO_PROTOCOL* is created.

Status Codes Returned

EFI_SUCCESS	The Bluetooth device is connected successfully.
EFI_ALREADY_STARTED	The Bluetooth device is already connected.
EFI_NOT_FOUND	The Bluetooth device is not found.
EFI_DEVICE_ERROR	A hardware error occurred trying to connect the Bluetooth device.

26.3.5 BLUETOOTH_CONFIG_PROTOCOL.Disconnect

Summary

Disconnect a Bluetooth device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_DISCONNECT)(
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL        *This,
  IN BLUETOOTH_ADDRESS                    *BD_ADDR,
  IN UINT8 *Reason
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

BD_ADDR
The address of Bluetooth device to be connected.

Reason
Bluetooth disconnect reason. See Bluetooth specification for detail.

Description

The Disconnect() function disconnects a Bluetooth device. When this function is returned successfully, the *EFI_BLUETOOTH_IO_PROTOCOL* associated with this device is destroyed and all services associated are stopped.

Status Codes Returned

EFI_SUCCESS	The Bluetooth device is disconnected successfully.
EFI_NOT_STARTED	The Bluetooth device is not connected.
EFI_NOT_FOUND	The Bluetooth device is not found.
EFI_DEVICE_ERROR	A hardware error occurred trying to disconnect the Bluetooth device.

26.3.6 BLUETOOTH_CONFIG_PROTOCOL.GetData

Summary

Get Bluetooth configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_GET_DATA) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL           *This,
  IN EFI_BLUETOOTH_CONFIG_DATA_TYPE          DataType,
  IN OUT UINTN                               *DataSize,
  IN OUT VOID                                *Data
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

DataType
Configuration data type.

DataSize
On input, indicates the size, in bytes, of the data buffer specified by *Data*. On output, indicates the amount of data actually returned.

Data
A pointer to the buffer of data that will be returned.

Description

The GetData() function returns Bluetooth configuration data. For remote Bluetooth device configuration data, please use *GetRemoteData()* function with valid BD_ADDR.

Related Definitions

```
typedef enum {
  EfiBluetoothConfigDataTypeDeviceName,         /* Relevant for LE*/
  EfiBluetoothConfigDataTypeClassOfDevice,
  EfiBluetoothConfigDataTypeRemoteDeviceState,  /* Relevant for LE*/
  EfiBluetoothConfigDataTypeSdpInfo,
  EfiBluetoothConfigDataTypeBDADDR,             /* Relevant for LE*/
  EfiBluetoothConfigDataTypeDiscoverable,       /* Relevant for LE*/
  EfiBluetoothConfigDataTypeControllerStoredPairedDeviceList,
  EfiBluetoothConfigDataTypeAvailableDeviceList,
  EfiBluetoothConfigDataTypeRandomAddress,      /* Relevant for LE*/
  EfiBluetoothConfigDataTypeRSSI,               /* Relevant for LE*/
  EfiBluetoothConfigDataTypeAdvertisementData,  /* Relevant for LE*/
  EfiBluetoothConfigDataTypeIoCapability,       /* Relevant for LE*/
  EfiBluetoothConfigDataTypeOOBDataFlag,        /* Relevant for LE*/
  EfiBluetoothConfigDataTypeKeyType,            /* Relevant for LE*/
  EfiBluetoothConfigDataTypeEncKeySize,         /* Relevant for LE*/
  EfiBluetoothConfigDataTypeMax,
}   EFI_BLUETOOTH_CONFIG_DATA_TYPE;
```

EfiBluetoothConfigDataTypeAdvertisementDataReport Advertisement report. Data structure is *UNIT8[]*.

EfiBluetoothConfigDataTypeKeyType
KeyType of Authentication Requirements flag of local device as *UINT8,* indicating requested security properties. See Bluetooth specification 3.H.3.5.1. BIT0: MITM, BIT1: SC.

EfiBluetoothConfigDataTypeDeviceName
Local/Remote Bluetooth device name. Data structure is zero terminated *CHAR8[]*.

EfiBluetoothConfigDataTypeClassOfDevice
nLocal/Remote Bluetooth device ClassOfDevice. Data structure is *BLUETOOTH_CLASS_OF_DEVICE*.

EfiBluetoothConfigDataTypeRemoteDeviceState
Remove Bluetooth device state. Data structure is *EFI_BLUETOOTH_CONFIG_REMOTE_DEVICE_STATE_TYPE*.

EfiBluetoothConfigDataTypeSdpInfo
Local/Remote Bluetooth device SDP information. Data structure is *UINT8[]*.

EfiBluetoothConfigDataTypeBDADDR
Local Bluetooth device address. Data structure is *BLUETOOTH_ADDRESS*.

EfiBluetoothConfigDataTypeDiscoverable
Local Bluetooth discoverable state. Data structure is *UINT8*. (Page scan and/or Inquiry scan)

EfiBluetoothConfigDataTypeControllerStoredPairedDeviceList
Local Bluetooth controller stored paired device list. Data structure is *BLUETOOTH_ADDRESS[]*.

EfiBluetoothConfigDataTypeAvailableDeviceList
Local available device list. Data structure is *BLUETOOTH_ADDRESS[]*.

```
typedef EFI_BLUETOOTH_CONFIG_REMOTE_DEVICE_STATE_TYPE UINT32;
#define EFI_BLUETOOTH_CONFIG_REMOTE_DEVICE_STATE_CONNECTED 0x1
#define EFI_BLUETOOTH_CONFIG_REMOTE_DEVICE_STATE_PAIRED 0x2

#define BLUETOOTH_HCI_LINK_KEY_SIZE 16
```

Status Codes Returned

EFI_SUCCESS	The Bluetooth configuration data is returned successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • DataSize is NULL. • * DataSize is not 0 and Data is NULL
EFI_UNSUPPORTED	The DataType is unsupported.
EFI_NOT_FOUND	The DataType is not found.
EFI_BUFFER_TOO_SMALL	The buffer is too small to hold the buffer. DataSize has been updated with the size needed to complete the request.

26.3.7 BLUETOOTH_CONFIG_PROTOCOL.SetData

Summary

Set Bluetooth configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_SET_DATA) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL        *This,
  IN EFI_BLUETOOTH_CONFIG_DATA_TYPE       DataType,
  IN UINTN                                DataSize,
  IN VOID                                 *Data
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

DataType
Configuration data type.

DataSize
Indicates the size, in bytes, of the data buffer specified by *Data*.

Data
A pointer to the buffer of data that will be set.

Description

The SetData() function sets local Bluetooth device configuration data. Not all DataType can be set.

Status Codes Returned

EFI_SUCCESS	The Bluetooth configuration data is set successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataSize* is 0. • *Data* is NULL.
EFI_UNSUPPORTED	The *DataType* is unsupported.
EFI_WRITE_PROTECTED	Cannot set configuration data.

26.3.8 BLUETOOTH_CONFIG_PROTOCOL.GetRemoteData

Summary

Get remove Bluetooth device configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_GET_REMOTE_DATA) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL           *This,
  IN EFI_BLUETOOTH_CONFIG_DATA_TYPE          DataType,
  IN BLUETOOTH_ADDRESS                       *BDAddr,
  IN OUT UINTN                               *DataSize,
  IN OUT VOID                                *Data
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

DataType
Configuration data type.

BDAddr
Remote Bluetooth device address.

DataSize
On input, indicates the size, in bytes, of the data buffer specified by *Data*. On output, indicates the amount of data actually returned.

Data
A pointer to the buffer of data that will be returned.

Description

The GetRemoteData() function returns remote Bluetooth device configuration data.

Status Codes Returned

EFI_SUCCESS	The remote Bluetooth device configuration data is returned successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is TRUE: • DataSize is NULL. • * DataSize is not 0 and Data is NULL
EFI_UNSUPPORTED	The DataType is unsupported.
EFI_NOT_FOUND	The DataType is not found.
EFI_BUFFER_TOO_SMALL	The buffer is too small to hold the buffer. DataSize has been updated with the size needed to complete the request.

26.3.9 BLUETOOTH_CONFIG_PROTOCOL.RegisterPinCallback

Summary

Register PIN callback function.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_REGISTER_PIN_CALLBACK) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL                      *This,
  IN EFI_BLUETOOTH_CONFIG_REGISTER_PIN_CALLBACK_FUNCTION *Callback,
  IN VOID                                               *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Callback
The callback function. *NULL* means unregister.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The RegisterPinCallback() function registers Bluetooth PIN callback function. The Bluetooth configuration driver must call *RegisterPinCallback()* to register a callback function. During pairing, Bluetooth bus driver must trigger this callback function, and Bluetooth configuration driver must handle callback function according to *CallbackType* during pairing. Both Legacy pairing and SSP (secure simple pairing) are required to be supported. See EFI_BLUETOOTH_PIN_CALLBACK_TYPE below for detail of each pairing mode.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_REGISTER_PIN_CALLBACK_FUNCTION)   (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL * *This,
  IN VOID * *Context,
```

(continues on next page)

(continued from previous page)

```
  IN EFI_BLUETOOTH_PIN_CALLBACK_TYPE *CallbackType,
  IN VOID * *InputBuffer,
  IN UINTN *InputBufferSize,
  OUT VOID ** *OutputBuffer,
  OUT UINTN * *OutputBufferSize
);
```

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Context
Context passed from registration.

CallbackType* Callback type in *EFI_BLUETOOTH_PIN_CALLBACK_TYPE*.

InBuffer
A pointer to the buffer of data that is input from callback caller.

InputBufferSize
Indicates the size, in bytes, of the data buffer specified by *InBuffer*.

OutputBuffer
A pointer to the buffer of data that will be output from callback callee. Callee allocates this buffer by using EFI Boot Service *AllocatePool()*.

OutputBufferSize*
Indicates the size, in bytes, of the data buffer specified by *OutputBuffer*.

```
typedef enum {
  EfiBluetoothCallbackTypeUserPasskeyNotification,
  EfiBluetoothCallbackTypeUserConfirmationRequest,
  EfiBluetoothCallbackTypeOOBDataRequest,
  EfiBluetoothCallbackTypePinCodeRequest,
  EfiBluetoothCallbackTypeMax,
} EFI_BLUETOOTH_PIN_CALLBACK_TYPE;
```

EfiBluetoothCallbackTypeUserPasskeyNotification
For SSP - passkey entry. Input buffer is Passkey (4 bytes). No output buffer. See Bluetooth HCI command for detail.

EfiBluetoothCallbackTypeUserConfirmationRequest
For SSP - just work and numeric comparison. Input buffer is numeric value (4 bytes). Output buffer is BOOLEAN (1 byte). See Bluetooth HCI command for detail.

EfiBluetoothCallbackTypeOOBDataRequest
For SSP - OOB. See Bluetooth HCI command for detail.

EfiBluetoothCallbackTypePinCodeRequest
For legacy paring. No input buffer. Output buffer is PIN code (<= 16 bytes). See Bluetooth HCI command for detail.

Status Codes Returned

EFI_SUCCESS	The PIN callback function is registered successfully.

26.3.10 BLUETOOTH_CONFIG_PROTOCOL.RegisterGetLinkKeyCallback

Summary

Register get link key callback function.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_REGISTER_GET_LINK_KEY_CALLBACK) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL                                  *This,
  IN EFI_BLUETOOTH_CONFIG_REGISTER_GET_LINK_KEY_CALLBACK_FUNCTION   Callback,
  IN VOID                                                           *Context
);
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Callback
The callback function. *NULL* means unregister.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The RegisterGetLinkKeyCallback() function registers Bluetooth get link key callback function. The Bluetooth configuration driver may call *RegisterGetLinkKeyCallback()* to register a callback function. When Bluetooth bus driver get Link_Key_Request_Event, Bluetooth bus driver must trigger this callback function if it is registered. Then the callback function in Bluetooth configuration driver must pass link key to Bluetooth bus driver. When the callback function is returned Bluetooth bus driver gets link key and must send HCI_Link_Key_Request_Reply to remote device. If this *GetLinkKey* callback function is not registered or Bluetooth configuration driver fails to return a valid link key, the Bluetooth bus driver must send HCI_Link_Key_Request_Negative_Reply to remote device. The original link key is passed by Bluetooth bus driver to Bluetooth configuration driver by using *EFI_BLUETOOTH_CONFIG_REGISTER_SET_LINK_KEY_CALLBACK_FUNCTION* . The Bluetooth configuration driver need save link key to a non-volatile safe place. (See Bluetooth specification, HCI_Link_Key_Request_Reply)

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI    *EFI_BLUETOOTH_CONFIG_REGISTER_GET_LINK_KEY_CALLBACK_FUNCTION)
  (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL  *This,
  IN VOID                           *Context,
  IN BLUETOOTH_ADDRESS              *BDAddr,
  OUT UINT8                         LinkKey[BLUETOOTH_HCI_LINK_KEY_SIZE]
  );
```

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Context
Context passed from registration.

CallbackType
Callback type in *EFI_BLUETOOTH_PIN_CALLBACK_TYPE*.

BDAddr
A pointer to Bluetooth device address.

LinkKey
A pointer to the buffer of link key.

Status Codes Returned

EFI_SUCCESS	The link key callback function is registered successfully.

26.3.11 BLUETOOTH_CONFIG_PROTOCOL.RegisterSetLinkKeyCallback

Summary

Register set link key callback function.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_REGISTER_SET_LINK_KEY_CALLBACK) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL       *This,
  IN EFI_BLUETOOTH_CONFIG_REGISTER_SET_LINK_KEY_CALLBACK_FUNCTION    *Callback,
  IN VOID                                *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Callback
The callback function. *NULL* means unregister.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The RegisterSetLinkKeyCallback() function registers Bluetooth link key callback function. The Bluetooth configuration driver may call *RegisterSetLinkKeyCallback()* to register a callback function to get link key from Bluetooth bus driver. When Bluetooth bus driver gets Link_Key_Notification_Event, Bluetooth bus driver must call this callback function if it is registered. Then the callback function in Bluetooth configuration driver must save link key to a safe place. This link key will be used by *EFI_BLUETOOTH_CONFIG_REGISTER_GET_LINK_KEY_CALLBACK_FUNCTION* later. (See Bluetooth specification, Link_Key_Notification_Event)

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI    *EFI_BLUETOOTH_CONFIG_REGISTER_SET_LINK_KEY_CALLBACK_FUNCTION) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL        *This,
  IN VOID                                 *Context,
  IN BLUETOOTH_ADDRESS                    *BDAddr,
```

(continues on next page)

(continued from previous page)

```
  IN UINT8                                  LinkKey[BLUETOOTH_HCI_LINK_KEY_SIZE]
  );
```

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Context
Context passed from registration.

CallbackType
Callback type in *EFI_BLUETOOTH_PIN_CALLBACK_TYPE*.

BDAddr
A pointer to Bluetooth device address.

LinkKey
A pointer to the buffer of link key.

Status Codes Returned

EFI_SUCCESS	The link key callback function is registered successfully.

26.3.12 BLUETOOTH_CONFIG_PROTOCOL.RegisterLinkConnectCompleteCallback

Summary

Register link connect complete callback function.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_CONFIG_REGISTER_CONNECT_COMPLETE_CALLBACK) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL                                *This,
  IN EFI_BLUETOOTH_CONFIG_REGISTER_CONNECT_COMPLETE_CALLBACK_FUNCTION  Callback,
  IN VOID                                                         *Context
);
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Callback
The callback function. *NULL* means unregister. to *CallbackType* defined by *EFI_BLUETOOTH_CONNECT_COMPLETE_CALLBACK_TYPE*.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The RegisterLinkConnectCompleteCallback() function registers Bluetooth link connect complete callback function. The Bluetooth Configuration driver may call *RegisterLinkConnectCompleteCallback()* to register a callback function. During pairing, Bluetooth bus driver must trigger this callback function to report device state, if it is registered. Then Bluetooth Configuration driver will get information on device connection, according

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI    *EFI_BLUETOOTH_CONFIG_REGISTER_CONNECT_COMPLETE_CALLBACK_FUNCTION) (
  IN EFI_BLUETOOTH_CONFIG_PROTOCOL                *This,
  IN VOID                                         *Context,
  IN EFI_BLUETOOTH_CONNECT_COMPLETE_CALLBACK_TYPE  CallbackType,
  IN BLUETOOTH_ADDRESS                            *BDAddr,
  IN VOID                                         *InputBuffer,
  IN UINTN                                        InputBufferSize
  );
```

This
Pointer to the *EFI_BLUETOOTH_CONFIG_PROTOCOL* instance.

Context
Context passed from registration.

CallbackType
Callback type in *EFI_BLUETOOTH_CONNECT_COMPLETE_CALLBACK_TYPE*.

BDAddr
A pointer to Bluetooth device address.

InputBuffer
A pointer to the buffer of data that is input from callback caller.

InputBufferSize
Indicates the size, in bytes, of the data buffer specified by *InputBuffer*.

```
typedef enum {
  EfiBluetoothConnCallbackTypeDisconnected,
  EfiBluetoothConnCallbackTypeConnected,
  EfiBluetoothConnCallbackTypeAuthenticated,
  EfiBluetoothConnCallbackTypeEncrypted,
} EFI_BLUETOOTH_CONNECT_COMPLETE_CALLBACK_TYPE;
```

EfiBluetoothConnCallbackTypeDisconnected
This callback is called when Bluetooth receive Disconnection_Complete event. Input buffer is Event Parameters of Disconnection_Complete Event defined in Bluetooth specification.

EfiBluetoothConnCallbackTypeConnected
This callback is called when Bluetooth receive Connection_Complete event. Input buffer is Event Parameters of Connection_Complete Event defined in Bluetooth specification.

EfiBluetoothConnCallbackTypeAuthenticated
This callback is called when Bluetooth receive Authentication_Complete event. Input buffer is Event Parameters of Authentication_Complete Event defined in Bluetooth specification.

EfiBluetoothConnCallbackTypeEncrypted
This callback is called when Bluetooth receive Encryption_Change event. Input buffer is Event Parameters of Encryption_Change Event defined in Bluetooth specification.

Status Codes Returned

EFI_SUCCESS	The link connect complete callback function is registered successfully.

26.4 EFI Bluetooth Attribute Protocol

26.4.1 EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL

Summary

This protocol provides service for Bluetooth ATT (Attribute Protocol) and GATT (Generic Attribute Profile) based protocol interfaces.

GUID

```
#define EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL_GUID \
  { 0x898890e9, 0x84b2, 0x4f3a, { 0x8c, 0x58, 0xd8, 0x57, 0x78, 0x13, 0xe0, 0xac }}
```

Protocol Interface Structure

„ code-block:

```
typedef struct  _EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL {
  EFI_BLUETOOTH_ATTRIBUTE_SEND_REQUEST          SendRequest;
  EFI_BLUETOOTH_ATTRIBUTE_REGISTER_FOR_SERVER_NOTIFICATION RegisterForServerNotification;
  EFI_BLUETOOTH_ATTRIBUTE_GET_SERVICE_INFO      GetServiceInfo;
  EFI_BLUETOOTH_ATTRIBUTE_GET_DEVICE_INFO       GetDeviceInfo;
} EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL;
```

Parameters

SendRequest
: Send a “REQUEST” or “COMMAND” message to remote server and receive a “RESPONSE” message for “REQUEST” from remote server according to Bluetooth attribute protocol data unit (PDU). See the SendRequest() function description.

RegisterForServerNotification
: Register or unregister a server-initiated PDU, such as “NOTIFICATION” or “INDICATION” on a characteristic value on remote server. See the RegisterForServerInitiatedMessage() function description.

GetServiceInfo
: Get discovered service data information from connected remote device. See GetServiceInfo() function description.

GetDeviceInfo
: Get the device information. See GetDeviceInfo() function description.

Description

The EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL provides services in ATT protocol and GATT profile. For detail of ATT protocol, and GATT profile, please refer to Bluetooth specification.

26.4.2 BLUETOOTH_ATTRIBUTE_PROTOCOL.SendRequest

Summary

Send a "REQUEST" or "COMMAND" message to remote server and receive a "RESPONSE" message for "REQUEST" from remote server according to Bluetooth attribute protocol data unit (PDU).

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_ATTRIBUTE_SEND_REQUEST)(
  IN EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL             *This,
  IN VOID                                         *Data,
  IN UINTN                                        DataLength,
  IN EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_FUNCTION    Callback,
  IN VOID                                         *Context
  );
```

Parameters

This
Pointer to the EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL instance.

Data
Data of a REQUEST or COMMAND message. The first byte is the attribute PDU related opcode, followed by opcode specific fields. See Bluetooth specification, Vol 3, Part F, Attribute Protocol.

DataLength
The length of Data in bytes.

Callback
Callback function to notify the RESPONSE is received to the caller, with the response buffer. Caller must check the response buffer content to know if the request action is success or fail. It may be NULL if the data is a COMMAND.

Context
Data passed into Callback function. It is optional parameter and may be NULL.

Description

The SendRequest() function sends a "REQUEST" or "COMMAND" message to remote server and receive a "RESPONSE" message for "REQUEST" from remote server according to Bluetooth attribute protocol data unit (PDU). In most cases, this interface is used to read attributes from remote device, or write attributes to remote device.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_FUNCTION) (
  IN EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL       *This,
  IN VOID                                   *Data,
  IN UINTN                                  DataLength,
  IN VOID                                   *Context
  );
```

This
Pointer to the EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL instance.

Data
Data received. The first byte is the attribute opcode, followed by opcode specific fields. See Bluetooth specification, Vol 3, Part F, Attribute Protocol. It might be a normal RESPONSE message, or ERROR RESPONSE message.

DataLength
The length of* Data *in bytes.*

Context
The context passed from the callback registration request.*

Status Codes Returned

EFI_SUCCESS	The request is sent successfully.
EFI_INVALID_PARAMETER	One or more parameters are invalid due to following conditions: • The Buffer is NULL. • The BufferLength is 0. • The opcode in Buffer is not a valid OPCODE according to Bluetooth specification. • The Callback is NULL.
EFI_DEVICE_ERROR	Sending the request failed due to the host controller or the device error.
EFI_NOT_READY	A GATT operation is already underway for this device
EFI_UNSUPPORTED	The attribute does not support the corresponding operation

26.4.3 BLUETOOTH_ATTRIBUTE_PROTOCOL.RegisterForServerNotification

Summary Register or unregister a server-initiated message, such as NOTIFICATION or INDICATION, on a characteristic value on remote server.

Prototype

code-block:

```
typedef
EFI_STATUS
(EFIAPI   *EFI_BLUETOOTH_ATTRIBUTE_REGISTER_FOR_SERVER_NOTIFICATION)
(
  IN EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL             *This,
  IN EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_PARAMETER   *CallbackParameter,
  IN EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_FUNCTION    Callback,
  IN VOID                                         *Context
);
```

Parameters

This
Pointer to the EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL instance.

CallbackParameter
The parameter of the callback.

Callback
Callback function for server-initiated attribute protocol. NULL callback function means unregister the server-initiated callback.

Context
Data passed into Callback function. It is optional parameter and may be NULL.

Description

The RegisterForServerNotification() function can be issued to request Bluetooth to register or unregister a server-initiated message, such as notification or indication, on a characteristic value on remote server. It can only be done if the characteristic supports that operation.

Related Definitions

```
typedef struct {
  UINT16               AttributeHandle;
}                      EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_PARAMETER_NOTIFICATION;
typedef struct {
  UINT16               AttributeHandle;
}                      EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_PARAMETER_INDICATION;
typedef struct {
  UINT32               Version;
  UINT8                AttributeOpCode;
  union {
    EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_PARAMETER_NOTIFICATION    Notification;
    EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_PARAMETER_INDICATION      Indication;
  } Parameter;
}                      EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_PARAMETER;
```

Version
The version of the structure. A value of zero represents the EFI_BLUETOOTH_ATTRIBUTE_CALLBACK_PARAMETER structure as defined here. Future version of this specification may extend this data structure in a backward compatible way and increase the value of Version.

AttributeOpCode
The attribute opcode for server-initiated attribute protocol. See Bluetooth specification, Vol 3, Part F, Attribute Protocol.

AttributeHandle
The attribute handle for notification or indication.

Status Codes Returned

EFI_SUCCESS	The callback function is registered or unregistered successfully
EFI_INVALID_PARAMETER	The attribute opcode is not server-initiated message opcode. See Bluetooth specification, Vol 3, Part F, Attribute Protocol.
EFI_ALREADY_STARTED	A callback function is already registered on the same attribute opcode and attribute handle, when the Callback is not NULL.
EFI_NOT_STARTED	A callback function is not registered on the same attribute opcode and attribute handle, when the Callback is NULL.
EFI_NOT_READY	A GATT operation is already underway for this device
EFI_UNSUPPORTED	The attribute does not support notification

26.4.4 BLUETOOTH_ATTRIBUTE_PROTOCOL.GetServiceInfo

Summary

Get Bluetooth discovered service information.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_ATTRIBUTE_GET_SERVICE_INFO)(
  IN EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL        *This,
  OUT UINTN                                  *ServiceInfoSize,
  OUT VOID                                   **ServiceInfo
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL* instance.

ServiceInfoSize
A pointer to the size, in bytes, of the *ServiceInfo* buffer.

ServiceInfo
A pointer to a callee allocated buffer that returns Bluetooth discovered service information. Callee allocates this buffer by using EFI Boot Service *AllocatePool()*.

Description

The GetServiceInfo() function returns Bluetooth discovered service information. The size of *ServiceInfo* structure should never be assumed and the value of *ServiceInfoSize* is the only valid way to know the size of *ServiceInfo*. The *ServiceInfo* buffer is a list Bluetooth service information structures defined below.

Related Definitions

```
typedef struct {
  UINT8           Length;
  union {
    UINT16        Uuid16;
    UINT32        Uuid32;
    UINT8         Uuid128[16];
  } Data;
} EFI_BLUETOOTH_UUID;
```

Length
The length of Bluetooth UUID data. The valid value is 2, 4, or 16.

Uuid16
The 16-bit Bluetooth UUID data.

Uuid32
The 32-bit Bluetooth UUID data.

Uuid128
The 128-bit Bluetooth UUID data.

```
typedef struct {
  EFI_BLUETOOTH_UUID                    Type;
  UINT16                                Length;
  UINT16                                AttributeHandle;
  EFI_BLUETOOTH_ATTRIBUTE_PERMISSION    AttributePermission;
}  EFI_BLUETOOTH_ATTRIBUTE_HEADER;
```

Type
The type of this structure. It must be EFI_BLUETOOTH_UUID. See Bluetooth GATT definition. Primary Service is 0x2800. Secondary Service is 0x2801. Include Service is 0x2802. Characteristic is 0x2803. Characteristic Descriptor is 0x2900.

Length
The length of this structure.

AttributeHandle
The handle of the service declaration. See Bluetooth specification.

AttributePermission
The permission of the attribute. This field is only valid for the attribute of the local device. This field should be ignored for the attribute of the remote device.

```
//
// Bluetooth Attribute Permission
//
typedef union {
 struct {
    UINT16     Readable : 1;
    UINT16     ReadEncryption : 1;
    UINT16     ReadAuthentication : 1;
    UINT16     ReadAuthorization : 1;
    UINT16     ReadKeySize : 5;
    UINT16     Reserved1 : 7;
    UINT16     Writeable : 1;
    UINT16     WriteEncryption : 1;
    UINT16     WriteAuthentication : 1;
    UINT16     WriteAuthorization : 1;
    UINT16     WriteKeySize : 5;
    UINT16     Reserved2 : 7;
 } Permission;
 UINT32 Data32;
} EFI_BLUETOOTH_ATTRIBUTE_PERMISSION;
```

Readable
The attribute is readable.

ReadEncryption
The encryption is required on read.

ReadAuthentication
The authentication is required on read.

ReadAuthorization
The authorization is required on read.

ReadKeySize
The size of key in bytes on read.

Writeable
The attribute is writeable.

WriteEncryption
The encryption is required on write.

WriteAuthentication
The authentication is required on write.

WriteAuthorization
The authorization is required on write.

WriteKeySize
The size of key in bytes on write.

```
typedef struct {
   EFI_BLUETOOTH_ATTRIBUTE_HEADER  Header;
   UINT16                          EndGroupHandle;
   EFI_BLUETOOTH_UUID              ServiceUuid;
}   EFI_BLUETOOTH_GATT_PRIMARY_SERVICE_INFO;
```

EndGroupHandle
The handle of the last attribute within the service definition. See Bluetooth specification.

Header
The header of this structure.

```
typedef struct {
  EFI_BLUETOOTH_ATTRIBUTE_HEADER     Header;
  UINT16                             StartGroupHandle;
  UINT16                             EndGroupHandle;
  EFI_BLUETOOTH_UUID                 ServiceUuid;
}  EFI_BLUETOOTH_GATT_INCLUDE_SERVICE_INFO;
```

Header
The header of this structure.

```
typedef struct {
 EFI_BLUETOOTH_ATTRIBUTE_HEADER       Header;
 UINT8                                CharacteristicProperties;
 UINT16                               CharacteristicValueHandle;
 EFI_BLUETOOTH_UUID                   CharacteristicUuid;
} EFI_BLUETOOTH_GATT_CHARACTERISTIC_INFO;
```

Header
The header of this structure.

```
typedef struct {
  EFI_BLUETOOTH_ATTRIBUTE_HEADER         Header;
  EFI_BLUETOOTH_UUID                     CharacteristicDescriptorUuid;
}  EFI_BLUETOOTH_GATT_CHARACTERISTIC_DESCRIPTOR_INFO;
```

Header
The header of this structure.

Status Codes Returned

EFI_SUCCESS	The Bluetooth discovered service information is returned successfully.
EFI_DEVICE_ERROR	A hardware error occurred trying to retrieve the Bluetooth discovered service information.

26.4.5 BLUETOOTH_ATTRIBUTE_PROTOCOL.GetDeviceInfo

Summary

Get Bluetooth device information.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_ATTRIBUTE_GET_DEVICE_INFO)(
  IN EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL      *This,
  OUT UINTN                                *DeviceInfoSize,
  OUT VOID                                 **DeviceInfo
);
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL* instance.

DeviceInfoSize
A pointer to the size, in bytes, of the *DeviceInfo* buffer.

DeviceInfo
A pointer to a callee allocated buffer that returns Bluetooth device information. Callee allocates this buffer by using EFI Boot Service *AllocatePool()*. If this device is Bluetooth classic device, EFI_BLUETOOTH_DEVICE_INFO should be used. If this device is Bluetooth LE device, EFI_BLUETOOTH_LE_DEVICE_INFO should be used.

Description

The GetDeviceInfo() function returns Bluetooth device information. The size of *DeviceInfo* structure should never be assumed and the value of *DeviceInfoSize* is the only valid way to know the size of *DeviceInfo*.

Related Definitions

```
typedef struct {
  UINT8                Address[6];
  UINT8                Type;
} BLUETOOTH_LE_ADDRESS;


typedef struct {
  UINT32                 Version;
  BLUETOOTH_LE_ADDRESS BD_ADDR;
  BLUETOOTH_LE_ADDRESS   DirectAddress;
  UINT8                  RSSI;
  UINTN                  AdvertismentDataSize;
  VOID                   *AdvertismentData;
}   EFI_BLUETOOTH_LE_DEVICE_INFO;
```

Version
The version of the structure. A value of zero represents th* e EFI_BLUETOOTH_LE_DEVICE_INFO structure as defined here. Future version of this specification may extend this data structure in a backward compatible way and increase the value of *Version*.

BD_ADDR
48bit Bluetooth device address and 1byte address type.

DirectAddress
48bit random device address and 1byte address type.

RSSI
Bluetooth RSSI. See Bluetooth specification for detail.

AdvertisementDataSize
The size of *AdvertisementData* in bytes.

AdvertisementData
Bluetooth LE advertisement data. See Bluetooth specification for detail.

Status Codes Returned

EFI_SUCCESS	The Bluetooth device information is returned successfully.
EFI_DEVICE_ERROR	A hardware error occurred trying to retrieve the Bluetooth device information.

26.4.6 EFI_BLUETOOTH_ATTRIBUTE_SERVICE_BINDING_PROTOCOL

Summary

The EFI Bluetooth ATTRIBUTE Service Binding Protocol is used to locate EFI Bluetooth ATTRIBUTE Protocol drivers to create and destroy child of the driver to communicate with other Bluetooth device by using Bluetooth ATTRIBUTE protocol.

GUID

```
#define EFI_BLUETOOTH_ATTRIBUTE_SERVICE_BINDING_PROTOCOL_GUID \
  { \
0x5639867a, 0x8c8e, 0x408d, 0xac, 0x2f, 0x4b, 0x61, 0xbd, 0xc0, 0xbb, 0xbb \
}
```

Description

The Bluetooth ATTRIBUTE consumer need locate EFI_BLUETOOTH_ATTRIBUTE_SERVICE_BINDING_PROTOCOL and call CreateChild() to create a new child of EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL instance. Then use EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL for Bluetooth communication. After use, the Bluetooth ATTRIBUTE consumer need call DestroyChild() to destroy it.

26.5 EFI Bluetooth LE Configuration Protocol

26.5.1 EFI_BLUETOOTH_LE_CONFIG_PROTOCOL

Summary

This protocol abstracts user interface configuration for BluetoothLe device.

GUID

```
#define EFI_BLUETOOTH_LE_CONFIG_PROTOCOL_GUID \
{ 0x8f76da58, 0x1f99, 0x4275, { 0xa4, 0xec, 0x47, 0x56, 0x51, 0x5b, 0x1c, 0xe8 }}
```

Protocol Interface Structure

```
typedef struct  _EFI_BLUETOOTH_LE_CONFIG_PROTOCOL {
  EFI_BLUETOOTH_LE_CONFIG_INIT Init;
  EFI_BLUETOOTH_LE_CONFIG_SCAN Scan;
  EFI_BLUETOOTH_LE_CONFIG_CONNECT Connect;
  EFI_BLUETOOTH_LE_CONFIG_DISCONNECT Disconnect;
  EFI_BLUETOOTH_LE_CONFIG_GET_DATA GetData;
  EFI_BLUETOOTH_LE_CONFIG_SET_DATA SetData;
  EFI_BLUETOOTH_LE_CONFIG_GET_REMOTE_DATA GetRemoteData;
EFI_BLUETOOTH_LE_CONFIG_REGISTER_SMP_AUTH_CALLBACK RegisterSmpAuthCallback;
  EFI_BLUETOOTH_LE_CONFIG_SEND_SMP_AUTH_DATA SendSmpAuthData;
  EFI_BLUETOOTH_LE_CONFIG_REGISTER_SMP_GET_DATA_CALLBACK    RegisterSmpGetDataCallback;
  EFI_BLUETOOTH_LE_CONFIG_REGISTER_SMP_SET_DATA_CALLBACK    RegisterSmpSetDataCallback;
EFI_BLUETOOTH_LE_CONFIG_REGISTER_CONNECT_COMPLETE_CALLBACK
RegisterLinkConnectCompleteCallback;
} EFI_BLUETOOTH_LE_CONFIG_PROTOCOL;
```

Parameters

Init
Initialize BluetoothLE host controller and local device. See the *Init()* function description.

Scan
Scan BluetoothLE device. See the *Scan()* function description.

Connect
Connect one BluetoothLE device. See the *Connect()* function description.

Disconnect
Disconnect one BluetoothLE device. See the *Disconnect()* function description.

GetData
Get BluetoothLE configuration data. See the *GetData()* function description.

SetData
Set BluetoothLE configuration data. See the *SetData()* function description.

GetRemoteData
Get remote BluetoothLE device data. See the *GetRemoteData()* function description.

RegisterSmpAuthCallback
Register Security Manager Callback function. This function will be called from Bluetooth BUS driver whenever user interaction is required for security protocol authorization/authentication. See the *RegisterSmpAuthCallback()* function description.

SendSmpAuthData
Send user input (Authentication/Authorization) such as passkey, confirmation (yes/no) in response to pairing request. See the *SendSmpAuthData()* function description.

RegisterSmpGetDataCallback
Register a callback function to get SMP related data. See the *RegisterSmpGetDataCallback()* function description.

RegisterSmpSetDataCallback
Register a callback function to set SMP related data. See the *RegisterSmpGetDataCallback()* function description.

RegisterLinkConnectCompleteCallback
Register link connect complete callback function. See the *RegisterLinkConnectCompleteCallback()* function description.

Description

The EFI_BLUETOOTH_LE_CONFIG_PROTOCOL abstracts the BluetoothLE configuration. User can use BluetoothLE configuration to interactive with BluetoothLE bus driver.

26.5.2 BLUETOOTH_LE_CONFIG_PROTOCOL.Init

Summary

Initialize BluetoothLE host controller and local device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_INIT)(
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL *This
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Description

The *Init()* function initializes BluetoothLE host controller and local device.

Status Codes Returned

EFI_SUCCESS	The BluetoothLE host controller and local device is initialized successfully.
EFI_DEVICE_ERROR	A hardware error occurred trying to initialize the BluetoothLE host controller and local device.

26.5.3 BLUETOOTH_LE_CONFIG_PROTOCOL.Scan

Summary

Scan BluetoothLE device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_SCAN)(
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL               *This,
  IN BOOLEAN                                        ReScan,
    IN UIN32                                        Timeout;
  IN EFI_BLUETOOTH_LE_CONFIG_SCAN_PARAMETER         *ScanParameter, OPTIONAL
  IN EFI_BLUETOOTH_LE_CONFIG_SCAN_CALLBACK_FUNCTION Callback,
  IN VOID                                           *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

ReScan
If *TRUE,* a new scan request is submitted no matter there is scan result before. If *FALSE* and there is scan result, the previous scan result is returned and no scan request is submitted.

Timeout
Duration in milliseconds for which to scan.

ScanParameter
If it is not *NULL,* the ScanParameter is used to perform a scan by the BluetoothLE bus driver. If it is NULL, the default parameter is used.

Callback
The callback function. This function is called if a BluetoothLE device is found during scan process.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL.*

Description

The Scan() function scans BluetoothLE device. When this function is returned, it just means scan request is submitted. It does not mean scan process is started or finished. Whenever there is a BluetoothLE device is found, the *Callback* function will be called. *Callback* function might be called before this function returns or after this function returns.

Related Definitions

```
typedef struct {
// Scan parameter
  UINT32                Version;
  UINT8                 ScanType;
  UINT16                ScanInterval;
  UINT16                ScanWindow;
  UINT8                 ScanningFilterPolicy;
```

(continues on next page)

(continued from previous page)

```
// Scan result filter
  UINT8                 AdvertisementFlagFilter;
}   EFI_BLUETOOTH_LE_CONFIG_SCAN_PARAMETER;
```

Version
The version of the structure. A value of zero represents the EFI_BLUETOOTH_LE_CONFIG_SCAN_PARAMETER structure as defined here. Future version of this specification may extend this data structure in a backward compatible way and increase the value of Version.

ScanType
Passive scanning or active scanning. See Bluetooth specification.

ScanInterval
Recommended scan interval to be used while performing scan.

ScanWindow
Recommended scan window to be used while performing a scan.

ScanningFilterPolicy
Recommended scanning filter policy to be used while performing a scan.

AdvertisementFlagFilter
This is one byte flag to serve as a filter to remove unneeded scan result. For example, set BIT0 means scan in LE Limited Discoverable Mode. Set BIT1 means scan in LE General Discoverable Mode. See Supplement to Bluetooth Core Specification.

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_SCAN_CALLBACK_FUNCTION) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL              *This,
  IN VOID                                          *Context,
  IN EFI_BLUETOOTH_LE_SCAN_CALLBACK_INFORMATION    *CallbackInfo
  );
```

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Context
Context passed from scan request.

CallbackInfo
Data related to scan result. *NULL* CallbackInfo means scan complete.

```
typedef struct{
  BLUETOOTH_LE_ADDRESS              BDAddr;
  BLUETOOTH_LE_ADDRESS              DirectAddress;
  UINT8                             RemoteDeviceState;
  INT8                              RSSI;
  UINTN                             AdvertisementDataSize;
  VOID                              *AdvertisementData;
}   EFI_BLUETOOTH_LE_SCAN_CALLBACK_INFORMATION;
```

Status Codes Returned

EFI_SUCCESS	The Bluetooth scan request is submitted.

continues on next page

Table 26.33 – continued from previous page

EFI_DEVICE_ERROR	A hardware error occurred trying to scan the Bluetooth device.

26.5.4 BLUETOOTH_LE_CONFIG_PROTOCOL.Connect

Summary

Connect a BluetoothLE device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_CONNECT)(
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL           *This,
  IN BOOLEAN                                    AutoReconnect,
  IN BOOLEAN                                    DoBonding;
  IN EFI_BLUETOOTH_LE_CONFIG_CONNECT_PARAMETER  *ConnectParameter, OPTIONAL
  IN BLUETOOTH_LE_ADDRESS                       *BD_ADDR
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

AutoReconnect
If *TRUE*. the BluetoothLE host controller needs to do an auto reconnect. If *FALSE,* the BluetoothLE host controller does not do an auto reconnect.

DoBonding
If *TRUE,* the BluetoothLE host controller needs to do a bonding. If *FALSE,* the BluetoothLE host controller does not do a bonding.

ConnectParameter
If it is not *NULL,* the ConnectParameter is used to perform a scan by the BluetoothLE bus driver. If it is NULL, the default parameter is used.

BD_ADDR
The address of the BluetoothLE device to be connected.

Description

The *Connect()* function connects a Bluetooth device. When this function is returned successfully, a new *EFI_BLUETOOTH_IO_PROTOCOL* is created.

Related Definitions

```
typedef struct {
  UINT32       Version;
  UINT16       ScanInterval;
  UINT16       ScanWindow;
  UINT16       ConnIntervalMin;
  UINT16       ConnIntervalMax;
  UINT16       ConnLatency;
  UINT16       SupervisionTimeout;
}   EFI_BLUETOOTH_LE_CONFIG_CONNECT_PARAMETER;
```

Version
The version of the structure. A value of zero represents the EFI_BLUETOOTH_LE_CONFIG_CONNECT_PARAMETER structure as defined here. Future version of this specification may extend this data structure in a backward compatible way and increase the value of Version.

ScanInterval
Recommended scan interval to be used while performing scan before connect.

ScanWindow
Recommended scan window to be used while performing a connection.

ConnIntervalMin
Minimum allowed connection interval. Shall be less than or equal to ConnIntervalMax.

ConnIntervalMax
Maximum allowed connection interval. Shall be greater than or equal to ConnIntervalMin.

ConnLatency
Slave latency for the connection in number of connection events.

SupervisionTimeout
Link supervision timeout for the connection.

Status Codes Returned

EFI_SUCCESS	The BluetoothLE device is connected successfully.
EFI_ALREADY_STARTED	The BluetoothLE device is already connected.
EFI_NOT_FOUND	The BluetoothLE device is not found.
EFI_DEVICE_ERROR	A hardware error occurred trying to connect the BluetoothLE device.

26.5.5 BLUETOOTH_LE_CONFIG_PROTOCOL.Disconnect

Summary

Disconnect a BluetoothLE device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_DISCONNECT)(
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL  *This,
  IN BLUETOOTH_LE_ADDRESS              *BD_ADDR,
  IN UINT8                             Reason
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

BD_ADDR
The address of BluetoothLE device to be connected.

Reason
BluetoothLE disconnect reason. See Bluetooth specification for detail.

Description

The Disconnect() function disconnects a BluetoothLE device. When this function is returned successfully, the *EFI_BLUETOOTH_ATTRIBUTE_PROTOCOL* associated with this device is destroyed and all services associated are stopped.

Status Codes Returned

EFI_SUCCESS	The BluetoothLE device is disconnected successfully.
EFI_NOT_STARTED	The BluetoothLE device is not connected.
EFI_NOT_FOUND	The BluetoothLE device is not found.
EFI_DEVICE_ERROR	A hardware error occurred trying to disconnect the BluetoothLE device.

26.5.6 BLUETOOTH_LE_CONFIG_PROTOCOL.GetData

Summary

Get BluetoothLE configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_GET_DATA) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL     *This,
  IN EFI_BLUETOOTH_CONFIG_DATA_TYPE       DataType,
  IN OUT UINTN                            *DataSize,
  IN OUT VOID                             *Data
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

DataType
Configuration data type.

DataSize
On input, indicates the size, in bytes, of the data buffer specified by *Data*. On output, indicates the amount of data actually returned.

Data
A pointer to the buffer of data that will be returned.

Description

The GetData() function returns BluetoothLE configuration data. For remote BluetoothLE device configuration data, please use *GetRemoteData()* function with valid BD_ADDR.

Status Codes Returned

EFI_SUCCESS	The BluetoothLE configuration data is returned successfully.

continues on next page

Table 26.36 – continued from previous page

EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataSize* is NULL. • * *DataSize* is 0. • *Data* is NULL.
EFI_UNSUPPORTED	The *DataType* is unsupported.
EFI_NOT_FOUND	The *DataType* is not found.
EFI_BUFFER_TOO_SMALL	The buffer is too small to hold the buffer.

26.5.7 BLUETOOTH_LE_CONFIG_PROTOCOL.SetData

Summary

Set BluetoothLE configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_SET_DATA) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL        *This,
  IN EFI_BLUETOOTH_CONFIG_DATA_TYPE          DataType,
  IN UINTN                                   DataSize,
  IN VOID                                    *Data
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

DataType
Configuration data type.

DataSize
Indicates the size, in bytes, of the data buffer specified by *Data*.

Data
A pointer to the buffer of data that will be set.

Description

The SetData() function sets local BluetoothLE device configuration data. Not all DataType can be set.

Status Codes Returned

EFI_SUCCESS	The BluetoothLE configuration data is set successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataSize* is 0. • *Data* is NULL.

continues on next page

Table 26.37 – continued from previous page

EFI_UNSUPPORTED	The *DataType* is unsupported.
EFI_WRITE_PROTECTED	Cannot set configuration data.

26.5.8 BLUETOOTH_LE_CONFIG_PROTOCOL.GetRemoteData

Summary

Get remove BluetoothLE device configuration data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_GET_REMOTE_DATA) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL     *This,
  IN EFI_BLUETOOTH_CONFIG_DATA_TYPE       DataType,
  IN BLUETOOTH_LE_ADDRESS                 *BDAddr,
  IN OUT UINTN                            *DataSize,
  IN OUT VOID                             *Data
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

DataType
Configuration data type.

BDAddr
Remote BluetoothLE device address.

DataSize
On input, indicates the size, in bytes, of the data buffer specified by *Data*. On output, indicates the amount of data actually returned.

Data
A pointer to the buffer of data that will be returned.

Description

The GetRemoteData() function returns remote BluetoothLE device configuration data.

Status Codes Returned

EFI_SUCCESS	The remote BluetoothLE device configuration data is returned successfully.
EFI_INVALID_PARAMETER	One or more of the following conditions is **TRUE**: • *DataSize* is NULL. • * *DataSize* is 0. • *Data* is NULL.
EFI_UNSUPPORTED	The *DataType* is unsupported.
EFI_NOT_FOUND	The *DataType* is not found.

continues on next page

Table 26.38 – continued from previous page

EFI_BUFFER_TOO_SMALL	The buffer is too small to hold the buffer.

26.5.9 BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterSmpAuthCallback

Summary

Register Security Manager Protocol callback function for user authentication/authorization.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_REGISTER_SMP_AUTH_CALLBACK)(
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL        *This,
  IN EFI_BLUETOOTH_LE_SMP_CALLBACK           Callback,
  IN VOID                                    *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Callback
Callback function for user authentication/authorization.

Context
Data passed into callback function. This is optional parameter and may be NULL.

Description

The RegisterSmpAuthCallback() function register Security Manager Protocol callback function for user authentication/authorization.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_SMP_CALLBACK) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL        *This,
  IN VOID                                    *Context,
  IN BLUETOOTH_LE_ADDRESS                    *BDAddr,
  IN EFI_BLUETOOTH_LE_SMP_EVENT_DATA_TYPE    EventDataType,
  IN UINTN                                   DataSize,
  IN VOID                                    *Data
  );
```

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Context
Data passed into callback function. This is optional parameter and may be NULL.

BDAddr
Remote BluetoothLE device address.

EventDataType
Event data type in *EFI_BLUETOOTH_LE_SMP_EVENT_DATA_TYPE*.

DataSize
Indicates the size, in bytes, of the data buffer specified by *Data*.

Data
A pointer to the buffer of data.

```
typedef enum {
  EfiBlutoothSmpAuthorizationRequestEvent,
  EfiBlutoothSmpPasskeyReadyEvent,
  EfiBlutoothSmpPasskeyRequestEvent,
  EfiBlutoothSmpOOBDataRequestEvent,
  EfiBlutoothSmpNumericComparisonEvent,
}   EFI_BLUETOOTH_LE_SMP_EVENT_DATA_TYPE;
```

EfiBlutoothSmpAuthorizationRequestEvent
It indicates an authorization request. No data is associated with the callback input. In the output data, the application should return the authorization value. The data structure is BOOLEAN. **TRUE** means YES. **FALSE** means NO.

EfiBlutoothSmpPasskeyReadyEvent
It indicates that a passkey has been generated locally by the driver, and the same passkey should be entered at the remote device. The callback input data is the passkey of type UINT32, to be displayed by the application. No output data should be returned.

EfiBlutoothSmpPasskeyRequestEvent
It indicates that the driver is requesting for the passkey has been generated at the remote device. No data is associated with the callback input. The output data is the passkey of type UINT32, to be entered by the user.

EfiBlutoothSmpOOBDataRequestEvent
It indicates that the driver is requesting for the passkey that has been pre-shared out-of-band with the remote device. No data is associated with the callback input. The output data is the stored OOB data of type UINT8[16].

EfiBlutoothSmpNumericComparisonEvent
In indicates that a number have been generated locally by the bus driver, and also at the remote device, and the bus driver wants to know if the two numbers match. The callback input data is the number of type UINT32. The output data is confirmation value of type BOOLEAN. **TRUE** means comparison pass. **FALSE** means comparison fail.

Status Codes Returned

EFI_SUCCESS	The SMP callback function is registered successfully.
EFI_ALREADY_STARTED	A callback function is already registered on the same attribute opcode and attribute handle, when the Callback is not NULL.
EFI_NOT_STARTED	A callback function is not registered on the same attribute opcode and attribute handle, when the Callback is NULL.

26.5.10 BLUETOOTH_LE_CONFIG_PROTOCOL.SendSmpAuthData

Summary

Send user authentication/authorization to remote device.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_SEND_SMP_AUTH_DATA)(
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL        *This,
  IN BLUETOOTH_LE_ADDRESS                    *BDAddr,
  IN EFI_BLUETOOTH_LE_SMP_EVENT_DATA_TYPE    EventDataType,
  IN UINTN                                   DataSize,
  IN VOID                                    *Data
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

BDAddr
Remote BluetoothLE device address.

EventDataType
Event data type in *EFI_BLUETOOTH_LE_SMP_EVENT_DATA_TYPE*.

DataSize
The size of *Data* in bytes, of the data buffer specified by *Data*.

Data
A pointer to the buffer of data that will be sent. The data format depends on the type of SMP event data being responded to. See *EFI_BLUETOOTH_LE_SMP_EVENT_DATA_TYPE*.

Description

The SendSmpAuthData() function sends user authentication/authorization to remote device. It should be used to send these information after the caller gets the request data from the callback function by RegisterSmpAuthCallback ().

Status Codes Returned

EFI_SUCCESS	The SMP authorization data is sent successfully.
EFI_NOT_READY	SMP is not in the correct state to receive the auth data

26.5.11 BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterSmpGetDataCallback

Summary

Register a callback function to get SMP related data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_BLUETOOTH_LE_CONFIG_REGISTER_SMP_GET_DATA_CALLBACK )(
```

(continues on next page)

(continued from previous page)

```
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL                *This,
  IN EFI_BLUETOOTH_LE_CONFIG_SMP_GET_DATA_CALLBACK   Callback,
  IN VOID                                            *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Callback
Callback function for SMP get data.

Context
Data passed into callback function. This is optional parameter and may be NULL.

Description

The RegisterSmpGetDataCallback() function registers a callback function to get SMP related data.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI * EFI_BLUETOOTH_LE_CONFIG_SMP_GET_DATA_CALLBACK) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL            *This,
  IN VOID                                        *Context,
  IN BLUETOOTH_LE_ADDRESS                        *BDAddr,
  IN EFI_BLUETOOTH_LE_SMP_DATA_TYPE              DataType,
  IN OUT UINTN                                   *DataSize,
  OUT VOID                                       *Data
  );
```

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Context
Data passed into callback function. This is optional parameter and may be NULL.

BDAddr
Remote BluetoothLE device address. For Local device setting, it should be NULL.

DataType
Data type in *EFI_BLUETOOTH_LE_SMP_DATA_TYPE*.

DataSize
On input, indicates the size, in bytes, of the data buffer specified by *Data*. On output, indicates the amount of data actually returned.

Data
A pointer to the buffer of data that will be returned.

```
typedef enum {
  // For local device only
  EfiBluetoothSmpLocalIR, /* If Key hierarchy is supported   */
  EfiBluetoothSmpLocalER, /* If Key hierarchy is supported   */
  EfiBluetoothSmpLocalDHK, /* If Key hierarchy is supported.  OPTIONAL */
```

(continues on next page)

(continued from previous page)

```
  // For peer specific
  EfiBluetoothSmpKeysDistributed = 0x1000,
  EfiBluetoothSmpKeySize,
  EfiBluetoothSmpKeyType,
  EfiBluetoothSmpPeerLTK,
  EfiBluetoothSmpPeerIRK,
  EfiBluetoothSmpPeerCSRK,
  EfiBluetoothSmpPeerRand,
  EfiBluetoothSmpPeerEDIV,
  EfiBluetoothSmpPeerSignCounter,
  EfiBluetoothSmpLocalLTK, /* If Key hierarchy not supported   */
  EfiBluetoothSmpLocalIRK, /* If Key hierarchy not supported   */
  EfiBluetoothSmpLocalCSRK, /* If Key hierarchy not supported   */
  EfiBluetoothSmpLocalSignCounter,
  EfiBluetoothSmpLocalDIV,
  EfiBluetoothSmpPeerAddressList,
  EfiBluetoothSmpMax,
}   EFI_BLUETOOTH_LE_SMP_DATA_TYPE;
```

EfiBlutoothSmpLocalIR
: It is a 128-bit Identity Root (IR) key to generate IRK. Data structure is UINT8[16]. See Bluetooth specification. This is only required when Bluetooth key hierarchy is supported. This type is for the local device only.

EfiBlutoothSmpLocalER
: It is a 128-bit Encryption Root (ER) key to generate LTK and CSRK. Data structure is UINT8[16]. See Bluetooth specification. This is only required when Bluetooth key hierarchy is supported. This type is for the local device only.

EfiBlutoothSmpLocalDHK
: It is a 128-bit Diversifier Hiding Key (DHK) to generate EDIV. Data structure is UINT8[16].See Bluetooth specification. This is only required when Bluetooth key hierarchy is supported. This type is for the local device only.

EfiBlutoothSmpKeysDistributed
: It is LE Key Distribution Format. Data structure is UINT8. See Bluetooth specification. This is the peer device specific information.

EfiBlutoothSmpKeySize
: It indicates the size of keys in bytes. It is the negotiated key size between local device and peer device. Data structure is UINTN. This is the peer device specific information.

EfiBlutoothSmpKeyType
: Indicates support for MITM/Secure connection. It is the negotiated Authentication Requirements between local device and peer device. See Bluetooth Spec 3.H.3.5.1. Data structure is UINT8. BIT0: MITM, BIT1: SC. This is the peer device specific information.

EfiBlutoothSmpPeerLTK
: It is a 128-bit Long-Term Key (LTK) to generate the contributory session key for an encrypted connection. Data structure is UINT8[16]. See Bluetooth specification. This is the peer device specific information.

EfiBlutoothSmpPeerIRK
: It is a 128-bit Identity Resolving Key (IRK) to generate and resolve random addresses. Data structure is UINT8[16]. See Bluetooth specification. This is the peer device specific information.

EfiBlutoothSmpPeerCSRK
: It is a 128-bit Connection-Signature Resolving Key (CSRK) to sign data and verify signatures on the receiving device. Data structure is UINT8[16]. See Bluetooth specification. This is the peer device specific information.

EfiBlutoothSmpPeerRand
It is a 64-bit Random number (Rand) to identify the LTK distributed during LE legacy pairing. Data structure is UINT64. See Bluetooth specification. This is the peer device specific information.

EfiBlutoothSmpPeerEDIV
It is a 16-bit Encrypted Diversifier (EDIV) to identify the LTK distributed during LE legacy pairing. Data structure is UINT16. See Bluetooth specification. This is the peer device specific information.

EfiBlutoothSmpPeerSignCounter
It is a 32-bit Sign Counter to assist MAC generation. Data structure is UINT32. See Bluetooth specification. This is the peer device specific information.

EfiBlutoothSmpLocakLTK
It is a 128-bit Long-Term Key (LTK) to generate the contributory session key for an encrypted connection. Data structure is UINT8[16]. See Bluetooth specification. This is only required when Bluetooth key hierarchy is not supported. This is the peer specific local device information.

EfiBlutoothSmpLocalIRK
It is a 128-bit Identity Resolving Key (IRK) to generate and resolve random addresses. Data structure is UINT8[16]. See Bluetooth specification. This is only required when Bluetooth key hierarchy is not supported. This is the peer specific local device information.

EfiBlutoothSmpLocalCSRK
It is a 128-bit Connection-Signature Resolving Key (CSRK) to sign data and verify signatures on the receiving device. Data structure is UINT8[16]. See Bluetooth specification. This is only required when Bluetooth key hierarchy is not supported. This is the peer specific local device information.

EfiBlutoothSmpLocalSignCounter
It is a 32-bit Sign Counter to assist MAC generation. Data structure is UINT32. See Bluetooth specification. This is the peer specific local device information.

EfiBlutoothSmpLocalDIV
It is a 16-bit Diversifier (DIV) to be used as index to recover LTK. Data structure is UINT16. See Bluetooth specification. This is the peer specific local device information.

EfiBluetoothSmpPeerAddressList
A list of Bluetooth peer addresses that have been connected before. The data structure is BLUETOOTH_LE_ADDRESS[]. The data size must be a multiple of sizeof(BLUETOOTH_LE_ADDRESS).

Status Codes Returned

EFI_SUCCESS	The SMP get data callback function is registered successfully.
EFI_ALREADY_STARTED	A callback function is already registered on the same attribute opcode and attribute handle, when the Callback is not NULL.
EFI_NOT_STARTED	A callback function is not registered on the same attribute opcode and attribute handle, when the Callback is NULL.

26.5.12 BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterSmpSetDataCallback

Summary

Register a callback function to set SMP related data.

Prototype

```
typedef
EFI_STATUS
(EFIAPI * EFI_BLUETOOTH_LE_CONFIG_REGISTER_SMP_SET_DATA_CALLBACK  )(
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL                  *This,
  IN EFI_BLUETOOTH_LE_CONFIG_SMP_SET_DATA_CALLBACK     Callback,
  IN VOID                                              *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Callback
Callback function for SMP set data.

Context
Data passed into callback function. This is optional parameter and may be NULL.

Description

The RegisterSmpSetDataCallback() function registers a callback function to set SMP related data.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI * EFI_BLUETOOTH_LE_CONFIG_SMP_SET_DATA_CALLBACK) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL                *This,
  IN VOID                                            *Context,
  IN BLUETOOTH_LE_ADDRESS                            *BDAddr,
  IN EFI_BLUETOOTH_LE_SMP_DATA_TYPE                  Type,
  IN UINTN                                           DataSize,
  IN VOID                                            *Data
  );
```

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Context
Data passed into callback function. This is optional parameter and may be NULL.

BDAddr
Remote BluetoothLE device address.

DataType
Data type in *EFI_BLUETOOTH_LE_SMP_DATA_TYPE*.

DataSize
Indicates the size, in bytes, of the data buffer specified by *Data*.

Data
A pointer to the buffer of data.

Status Codes Returned

EFI_SUCCESS	The SMP get data callback function is registered successfully.
EFI_ALREADY_STARTED	A callback function is already registered on the same attribute opcode and attribute handle, when the Callback is not NULL.
EFI_NOT_STARTED	A callback function is not registered on the same attribute opcode and attribute handle, when the Callback is NULL.

26.5.13 BLUETOOTH_LE_CONFIG_PROTOCOL.RegisterLinkConnectCompleteCallback

Summary

Register link connect complete callback function.

Prototype

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_REGISTER_CONNECT_COMPLETE_CALLBACK) (
  IN EFI_BLUETOOTH_LE_CONFIG_PROTOCOL                    *This,
  IN EFI_BLUETOOTH_LE_CONFIG_CONNECT_COMPLETE_CALLBACK   Callback,
  IN VOID                                                *Context
  );
```

Parameters

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Callback
The callback function. *NULL* means unregister.

Context
Data passed into *Callback* function. This is optional parameter and may be *NULL*.

Description

The RegisterLinkConnectCompleteCallback() function registers Bluetooth link connect complete callback function. The Bluetooth Configuration driver may call *RegisterLinkConnectCompleteCallback()* to register a callback function. During pairing, Bluetooth bus driver must trigger this callback function to report device state, if it is registered. Then Bluetooth Configuration driver will get information on device connection, according to *CallbackType* defined by EFI_BLUETOOTH_CONNECT_COMPLETE_CALLBACK_TYPE.

Related Definitions

```
typedef
EFI_STATUS
(EFIAPI *EFI_BLUETOOTH_LE_CONFIG_CONNECT_COMPLETE_CALLBACK) (
  IN  EFI_BLUETOOTH_LE_CONFIG_PROTOCOL               *This,
  IN  VOID                                           *Context,
  IN  EFI_BLUETOOTH_CONNECT_COMPLETE_CALLBACK_TYPE   CallbackType,
  IN  BLUETOOTH_LE_ADDRESS                           *BDAddr,
  IN  VOID                                           *InputBuffer,
  IN  UINTN                                          InputBufferSize
  );
```

This
Pointer to the *EFI_BLUETOOTH_LE_CONFIG_PROTOCOL* instance.

Context
Context passed from registration.

CallbackType
Callback type in *EFI_BLUETOOTH_CONNECT_COMPLETE_CALLBACK_TYPE*.

BDAddr
A pointer to BluetoothLE device address.

InputBuffer
A pointer to the buffer of data that is input from callback caller.

InputBufferSize
Indicates the size, in bytes, of the data buffer specified by *InputBuffer*.

Status Codes Returned

EFI_SUCCESS	The link connect complete callback function is registered successfully.
EFI_ALREADY_STARTED	A callback function is already registered on the same attribute opcode and attribute handle, when the Callback is not NULL.
EFI_NOT_STARTED	A callback function is not registered on the same attribute opcode and attribute handle, when the Callback is NULL.

2876 .. _network-protocols-vlan-eap-wi-fi-and-supplicant: